

# Worksheet — 2.3 Algorithms — ANSWER SHEET

Separate answer sheet / mark scheme — keep for marking.

## Answer key

### Activity 1 — Big O match

| Algorithm                    | Best          | Average       | Worst         |
|------------------------------|---------------|---------------|---------------|
| Linear search                | $O(1)$        | $O(n)$        | $O(n)$        |
| Binary search                | $O(1)$        | $O(\log n)$   | $O(\log n)$   |
| Bubble sort (with swap flag) | $O(n)$        | $O(n^2)$      | $O(n^2)$      |
| Insertion sort               | $O(n)$        | $O(n^2)$      | $O(n^2)$      |
| Merge sort                   | $O(n \log n)$ | $O(n \log n)$ | $O(n \log n)$ |
| Quick sort                   | $O(n \log n)$ | $O(n \log n)$ | $O(n^2)$      |

Note: bubble sort best case is  $O(n)$  **only with a swap flag** on already-sorted data. Quick sort worst case is  $O(n^2)$  with a poor pivot (e.g. already-sorted data, first/last element pivot).

### Activity 2 — Sort trace

**BUBBLE SORT** of [5, 1, 4, 2, 8] (compare adjacent pairs, swap if left > right; largest bubbles to the end each pass):

| Pass  | List state after this pass | Swaps this pass                                                                 |
|-------|----------------------------|---------------------------------------------------------------------------------|
| Start | 5 1 4 2 8                  | —                                                                               |
| 1     | 1 4 2 5 8                  | $5 \leftrightarrow 1$ , $5 \leftrightarrow 4$ , $5 \leftrightarrow 2$ = 3 swaps |
| 2     | 1 2 4 5 8                  | $4 \leftrightarrow 2$ = 1 swap                                                  |
| 3     | 1 2 4 5 8                  | 0 swaps → with swap flag, algorithm stops here                                  |

Final sorted list: 1 2 4 5 8

Pass-by-pass detail of pass 1 (each adjacent comparison): - (5,1) → swap → 1 5 4 2 8 - (5,4) → swap → 1 4 5 2 8 - (5,2) → swap → 1 4 2 5 8 - (5,8) → no swap → 1 4 2 5 8

**INSERTION SORT** alternative (acceptable if student chose insertion) — list after each pass (element inserted into sorted front):

| Pass         | List state after this pass |
|--------------|----------------------------|
| Start        | 5 1 4 2 8                  |
| 1 (insert 1) | 1 5 4 2 8                  |
| 2 (insert 4) | 1 4 5 2 8                  |
| 3 (insert 2) | 1 2 4 5 8                  |
| 4 (insert 8) | 1 2 4 5 8                  |

Final sorted list: 1 2 4 5 8 (same result).

### Activity 3 — Binary search for 27

List: indices 0–8, values 3, 7, 11, 15, 19, 23, 27, 31, 35.

| Step | low | high | mid (index) | value at mid | < = > target?                                                         |
|------|-----|------|-------------|--------------|-----------------------------------------------------------------------|
| 1    | 0   | 8    | 4           | 19           | $19 < 27 \rightarrow$ search right, $\text{low} = \text{mid} + 1 = 5$ |
| 2    | 5   | 8    | 6           | 27           | $27 = 27 \rightarrow$ FOUND                                           |

Found at index: **6**. Number of comparisons: **2**.

(Working: step 1  $\text{mid} = (0+8) \text{ DIV } 2 = 4$ ; step 2  $\text{mid} = (5+8) \text{ DIV } 2 = 6$ .)

### Activity 4 — Justifications

**(a)** Binary search is  $O(\log n)$  because **each comparison halves the size of the list still to be searched** — it discards the half that cannot contain the target. The number of times  $n$  can be halved before reaching 1 is  $\log_2(n)$ , so the number of comparisons grows logarithmically with  $n$ . (Doubling the list size adds only one extra comparison.)

**(b)** Bubble sort is  $O(n^2)$  because it uses **two nested loops**: an outer loop of up to  $n$  passes, and for each pass an inner loop making up to  $n$  adjacent comparisons. That gives roughly  $n \times n = n^2$  comparisons, so time grows with the square of the input size.

**(c)** Binary search requires the list to be **sorted**; linear search works on unsorted data.

### Activity 5 — Tree traversals

Tree: root 40; left subtree 20 (children 10, 30); right subtree 60 (children 50, 70).

- **Pre-order** (Node, Left, Right): 40, 20, 10, 30, 60, 50, 70
- **In-order** (Left, Node, Right): 10, 20, 30, 40, 50, 60, 70
- **Post-order** (Left, Right, Node): 10, 30, 20, 50, 70, 60, 40

Ascending order is produced by the **in-order** traversal of a BST.

### Activity 6 — Dijkstra A to F

Edges (undirected): A-B 4, A-C 2, B-C 1, B-D 5, C-D 8, C-E 10, D-E 2, D-F 6, E-F 3.

Settle the unvisited node with the smallest tentative distance each round; relax its neighbours.

| Settled (order) | A | B        | C        | D        | E        | F        |
|-----------------|---|----------|----------|----------|----------|----------|
| Initial         | 0 | $\infty$ | $\infty$ | $\infty$ | $\infty$ | $\infty$ |
| Settle A (0)    | 0 | 4        | 2        | $\infty$ | $\infty$ | $\infty$ |
| Settle C (2)    | 0 | 3        | 2        | 10       | 12       | $\infty$ |
| Settle B (3)    | 0 | 3        | 2        | 8        | 12       | $\infty$ |
| Settle D (8)    | 0 | 3        | 2        | 8        | 10       | 14       |
| Settle E (10)   | 0 | 3        | 2        | 8        | 10       | 13       |
| Settle F (13)   | 0 | 3        | 2        | 8        | 10       | 13       |

Relaxation notes: - From A: B = 4, C = 2. - Settle C (2): B =  $\min(4, 2+1) = 3$ ; D =  $2+8 = 10$ ; E =  $2+10 = 12$ .  
- Settle B (3): D =  $\min(10, 3+5) = 8$ ; (C already settled). - Settle D (8): E =  $\min(12, 8+2) = 10$ ; F =  $8+6 = 14$ .  
- Settle E (10): F =  $\min(14, 10+3) = 13$ . - Settle F (13): done.

**Shortest path A → F:** A → C → B → D → E → F

**Total distance: 13.**

(Check: A → C 2, C → B 1, B → D 5, D → E 2, E → F 3 =  $2+1+5+2+3 = 13$ . Compare alternative A → C → B → D → F =  $2+1+5+6 = 14$ , longer. So 13 is optimal.)

## Challenge

**(a)** A heuristic is **admissible** if it **never overestimates** the true shortest-distance from a node to the goal F. To check, compare each  $h(n)$  to the actual shortest distance to F (computed by Dijkstra from each node): - True shortest distances to F — F:0, E:3, D:5 (D → E → F =  $2+3$ ), B:10 (B → D → E → F =  $5+2+3$ ), C:11 (C → B → D → E → F =  $1+5+2+3$ ), A:13. - Heuristic given: A=9, B=7, C=8, D=4, E=3, F=0. Each is  $\leq$  the true distance ( $9 \leq 13$ ,  $7 \leq 10$ ,  $8 \leq 11$ ,  $4 \leq 5$ ,  $3 \leq 3$ ,  $0 \leq 0$ ), so the heuristic is **admissible** and A\* would find the optimal path.

**(b)** A explores fewer nodes than Dijkstra when a good (informative) heuristic directs the search toward the goal instead of expanding outward in all directions — useful when you only need the path to one goal node. A behaves **exactly like Dijkstra** when the heuristic is  **$h(n) = 0$**  for every node (no information), because then  $f(n) = g(n)$ .